

Introduction to Algorithm

기초 알고리즘

Backtracking 구현 및 실습

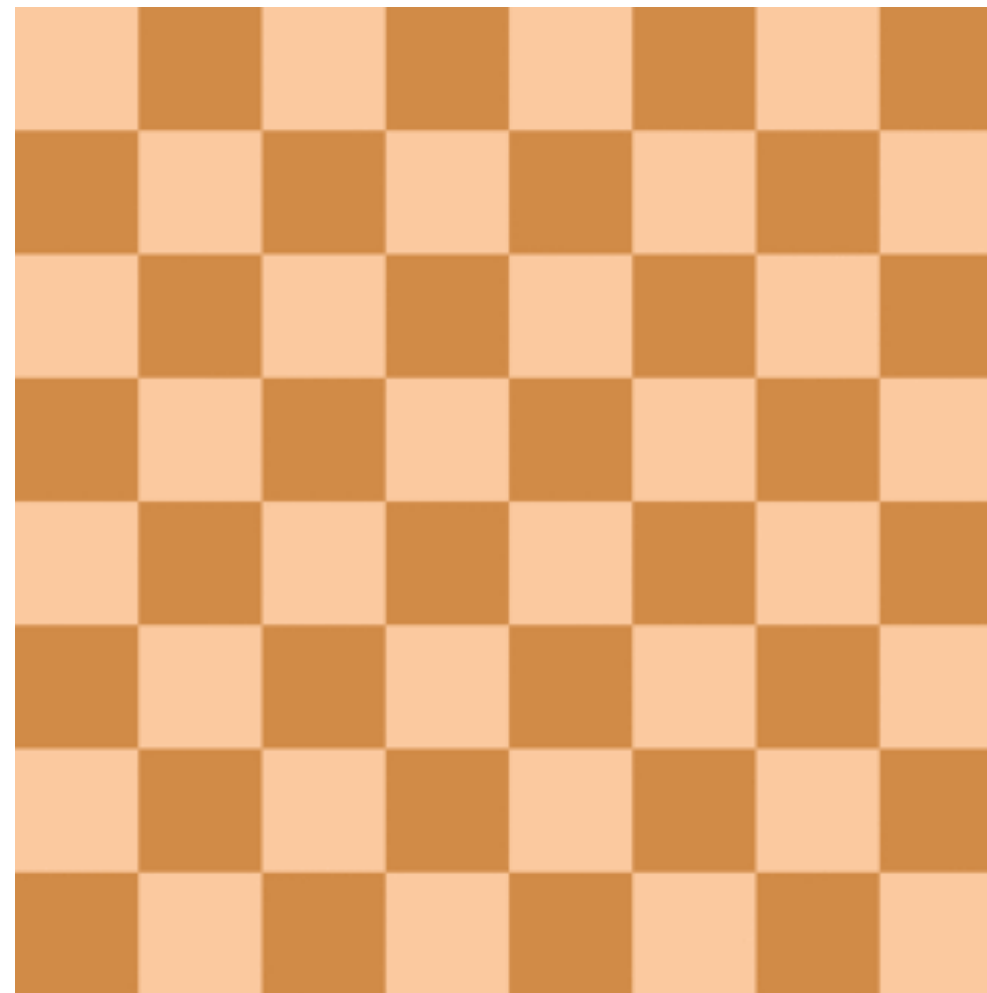


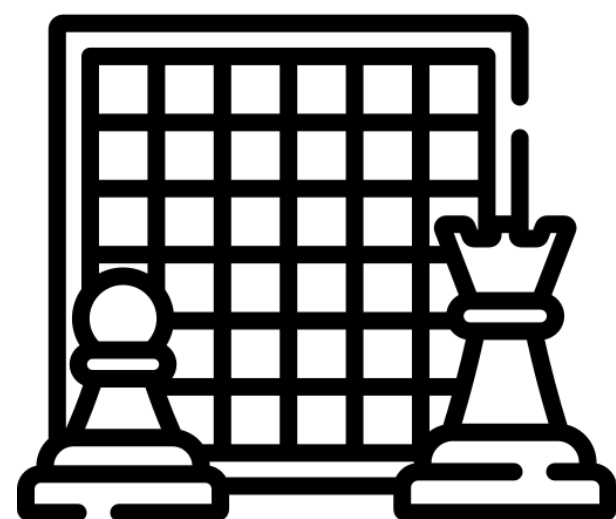
조금 복잡한 백트래킹 알고리즘 문제를 풀어보도록 하자



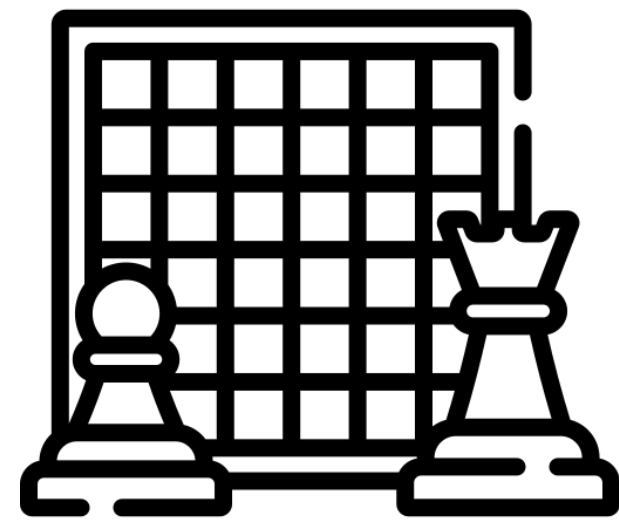
예시 - Nqueen

$N \times N$ 크기의 체스판에 N 개의 퀸을 서로 공격하지 못하게 놓아보자.
단, 퀸은 같은 행, 열, 대각선 위에 있는 말을 공격할 수 있다.

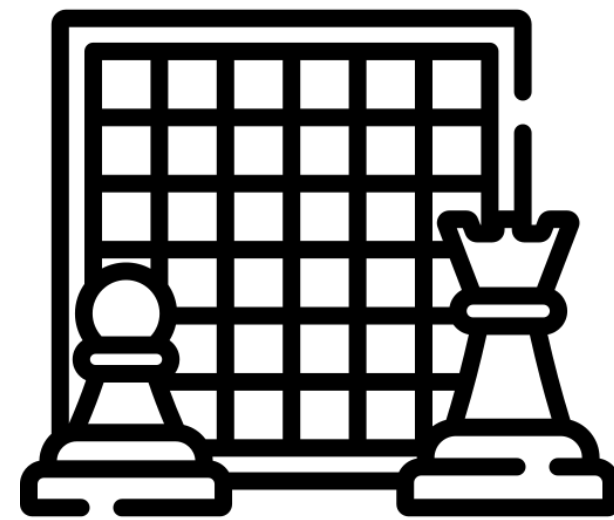




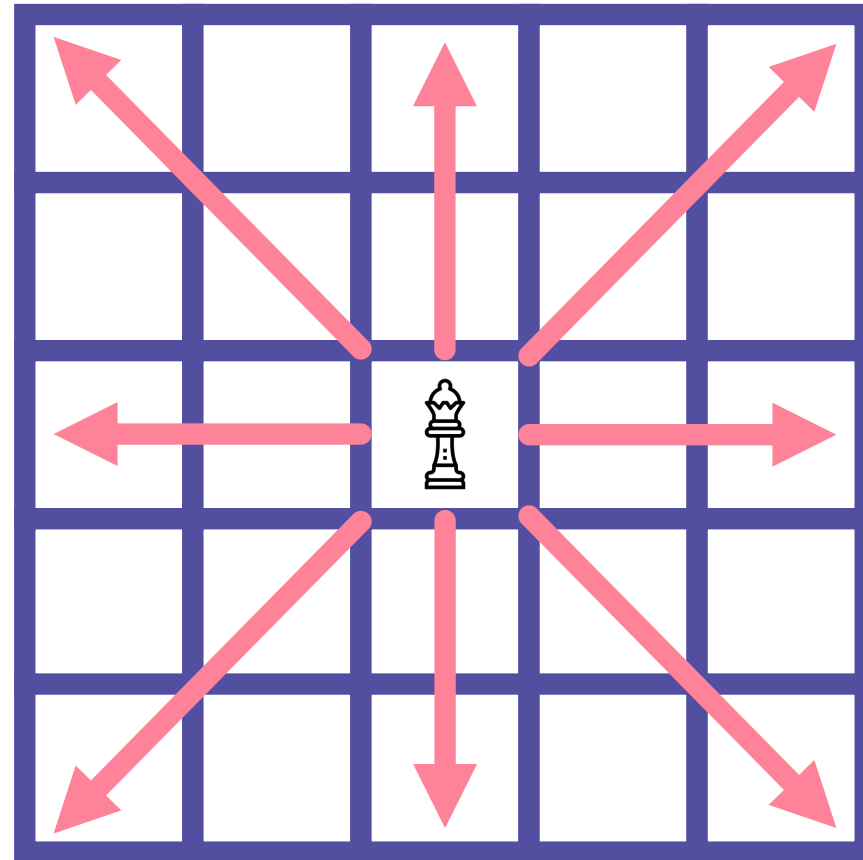
유명한 게임인 체스



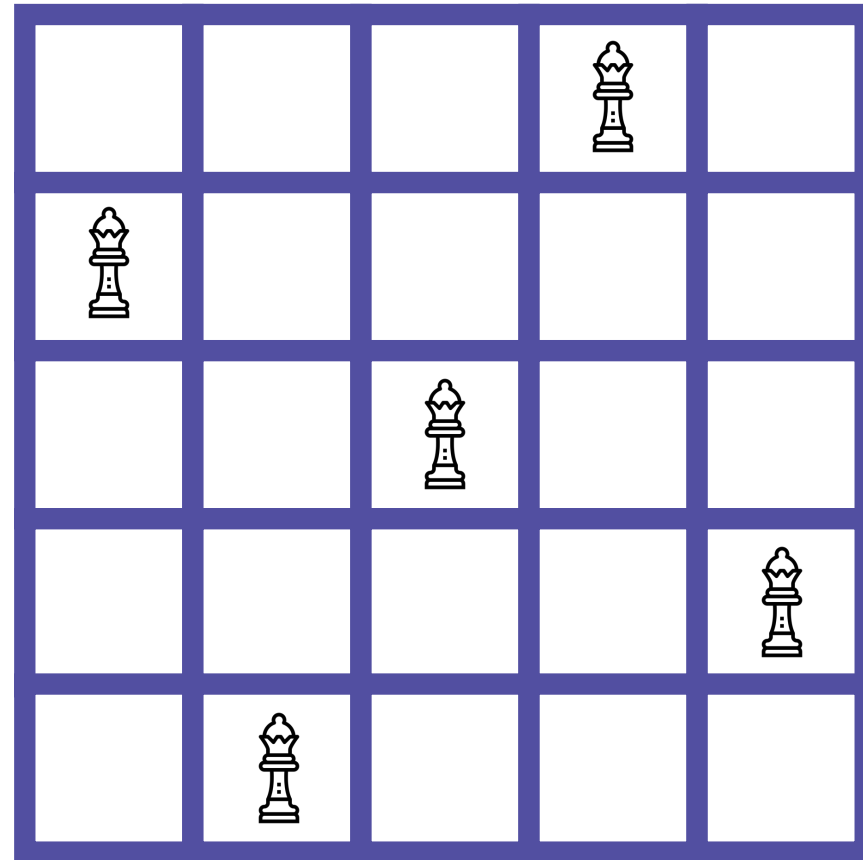
이 게임은 8×8 게임판에 놓인 다양한 말을 움직여
상대방의 킹을 잡으면 승리하는 게임



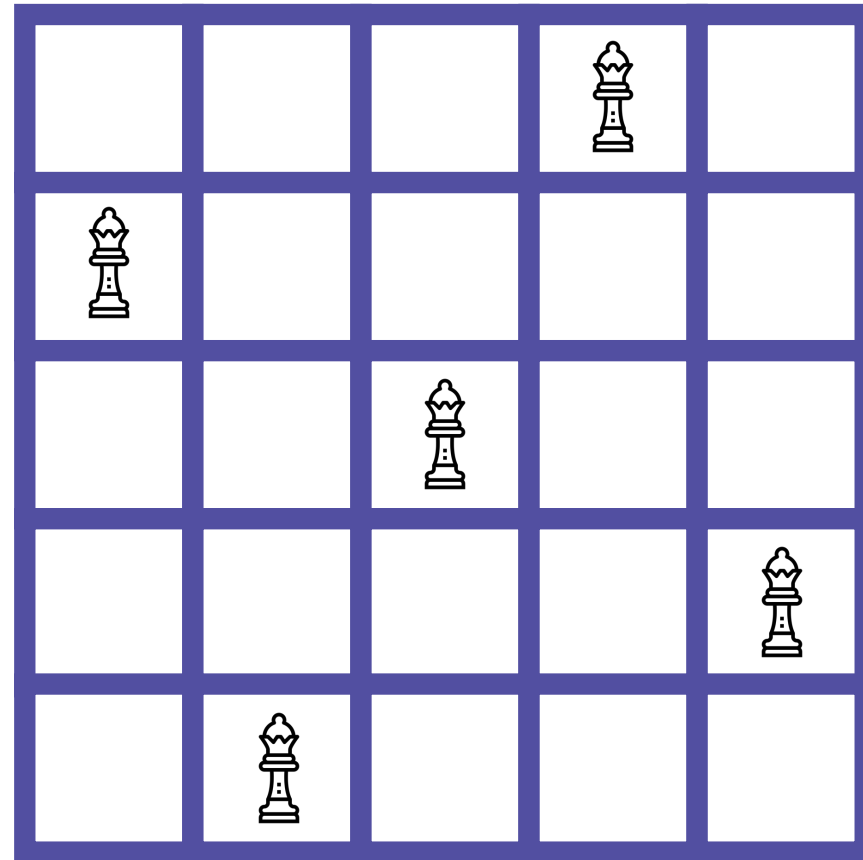
각각의 말은 모양에 따라 움직이는 방법이 다름



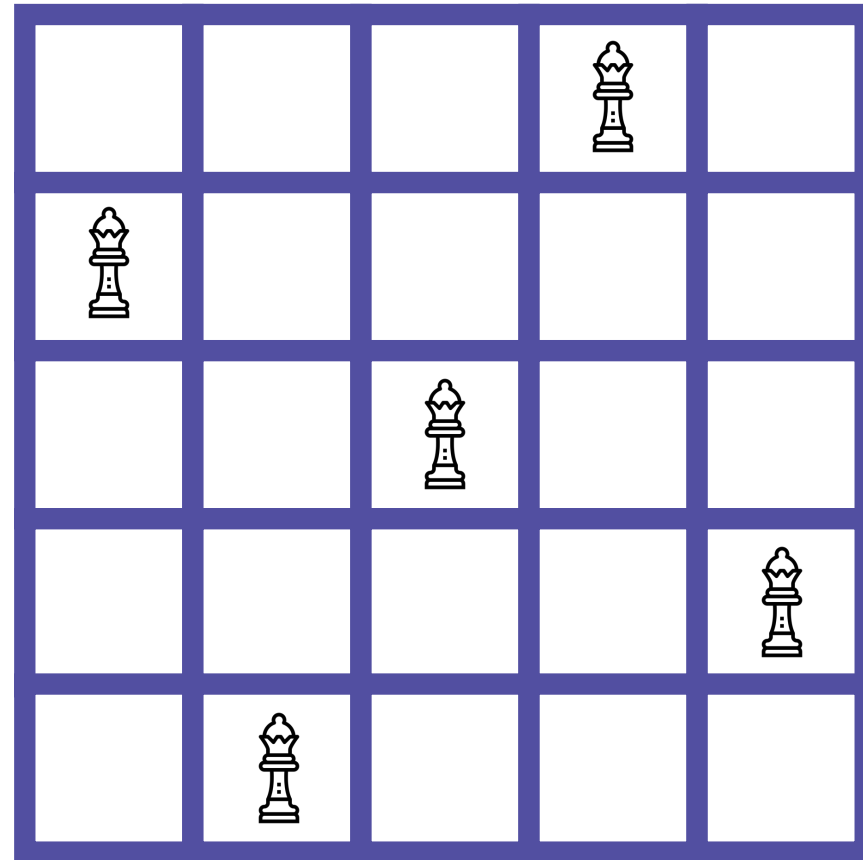
그 중, 퀸은 가로/세로/대각선 어느 방향으로든
거리에 상관없이 움직일 수 있는 강력한 말



다음과 같이 5×5 게임판에 퀸을 배치하면,
서로 공격할 수 없게 배치할 수 있음

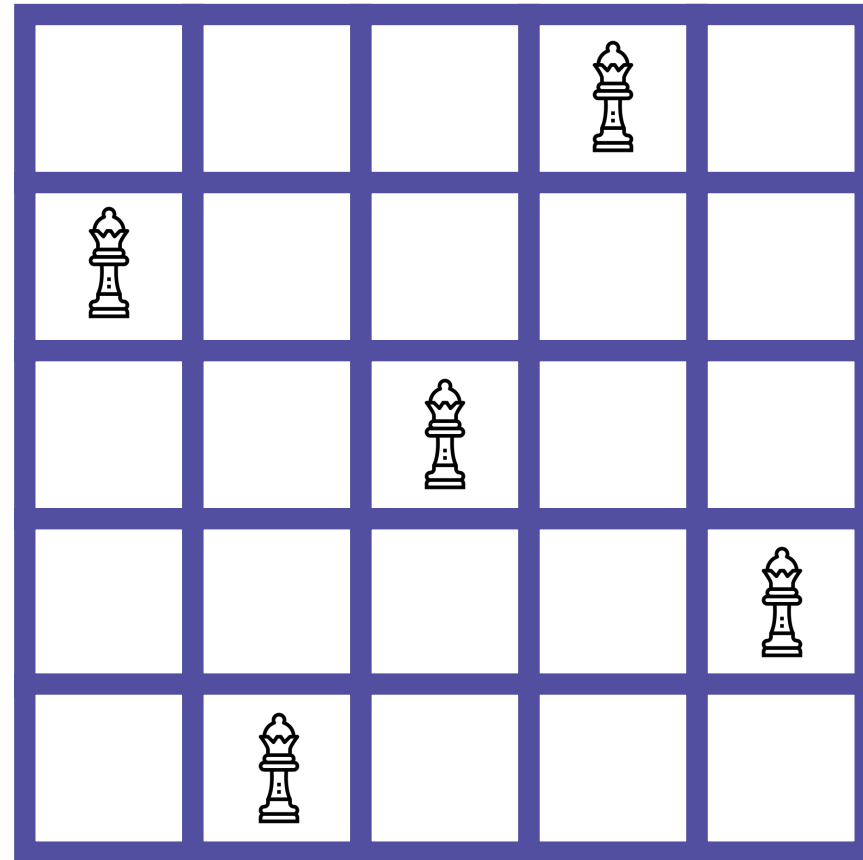


그렇다면, 보드의 한 변의 길이가 주어질 때,
퀸을 최대한 많이 배치할 수 있는 경우의 수를 구할 수 있을까?

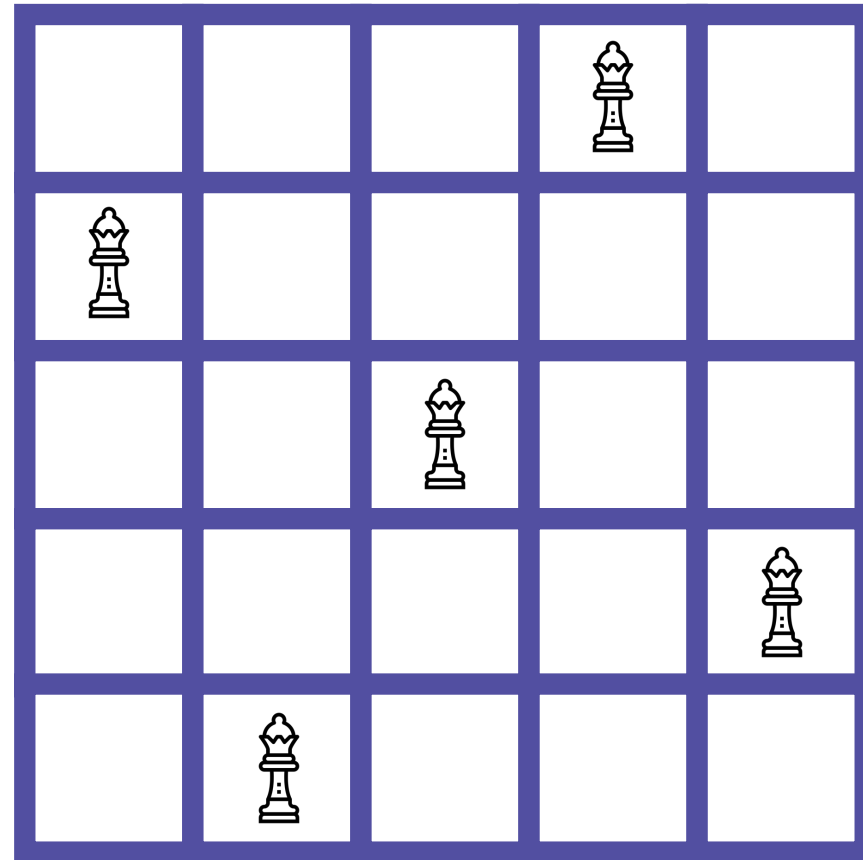


잘 생각해보면, 가로와 세로에는 퀸이 하나 밖에 올 수 없으니,

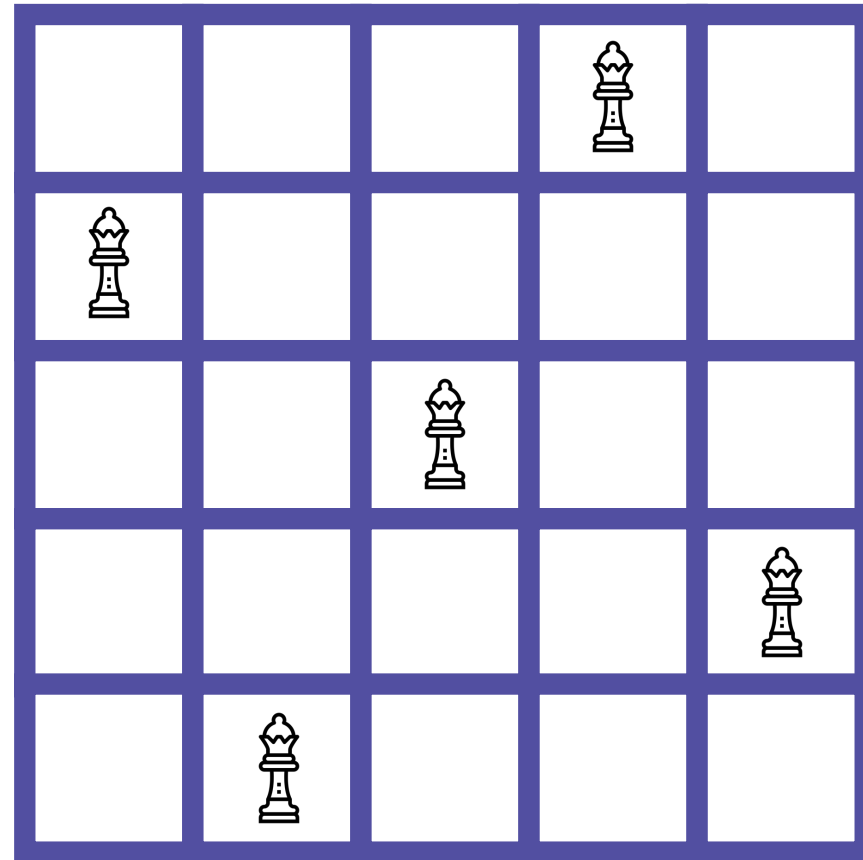
$N \times N$ 크기의 게임판엔 최대 N 개의 퀸을 둘 수 있을 것



완전탐색으로 구한다면 시간복잡도는 어떻게 될까?

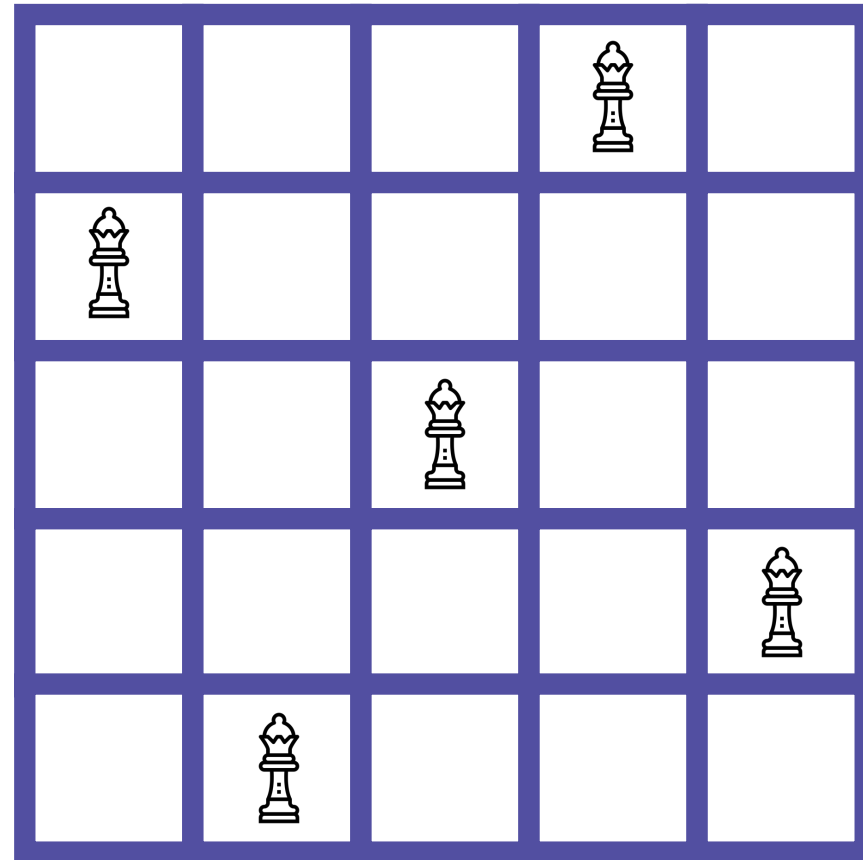


전체 칸은 N^2 개 이고, N 개의 퀸에 대해
모든 조합을 시도해야 함

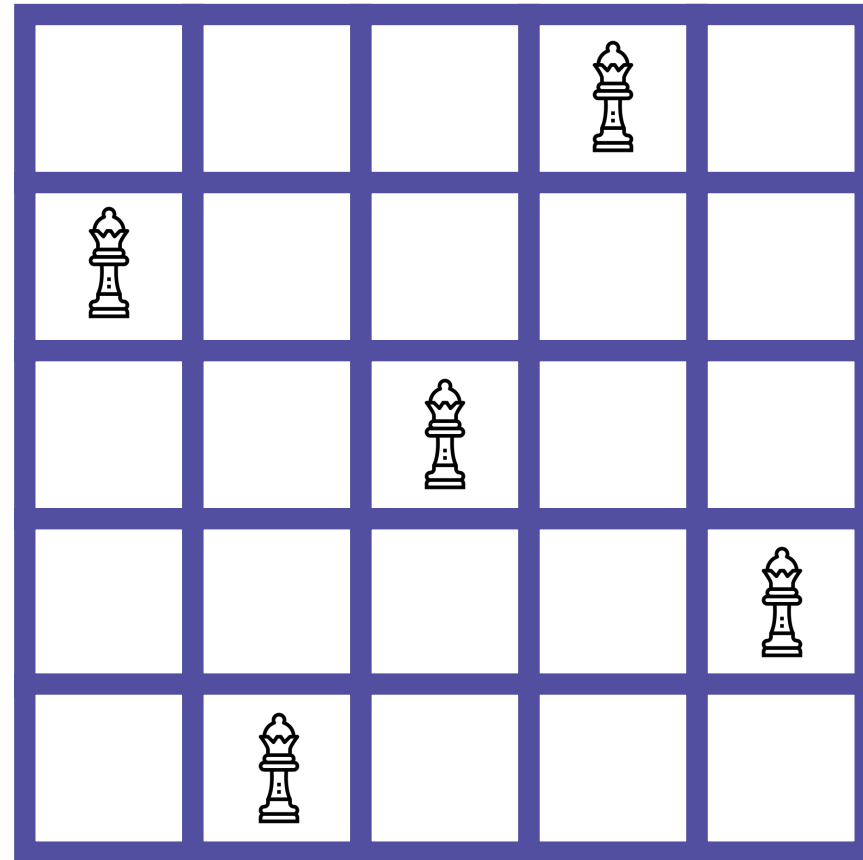


즉, 시간복잡도는 대략

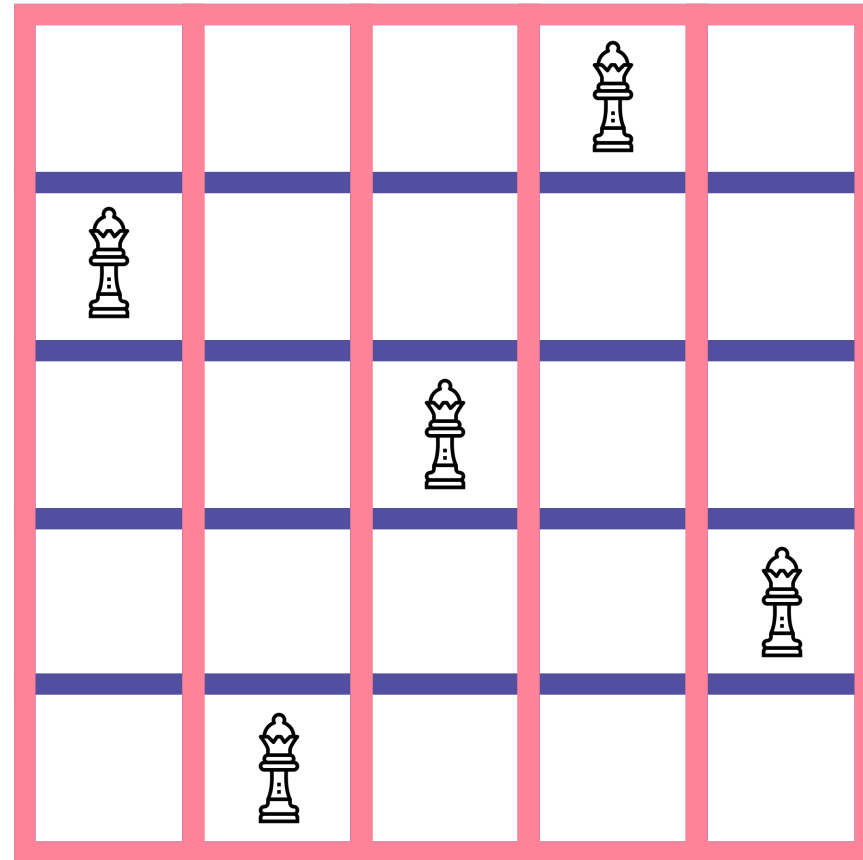
$$O(N^2 \times N^2 \times N^2 \times \dots) = O(N^{2N}) \text{ 이 될 것}$$



N이 5만 되어도, 약 5^{10} 번의 연산이 진행된다는 이야기



효율적으로 문제를 해결하기 위해,
문제를 보는 관점을 조금 바꿔보도록 하자

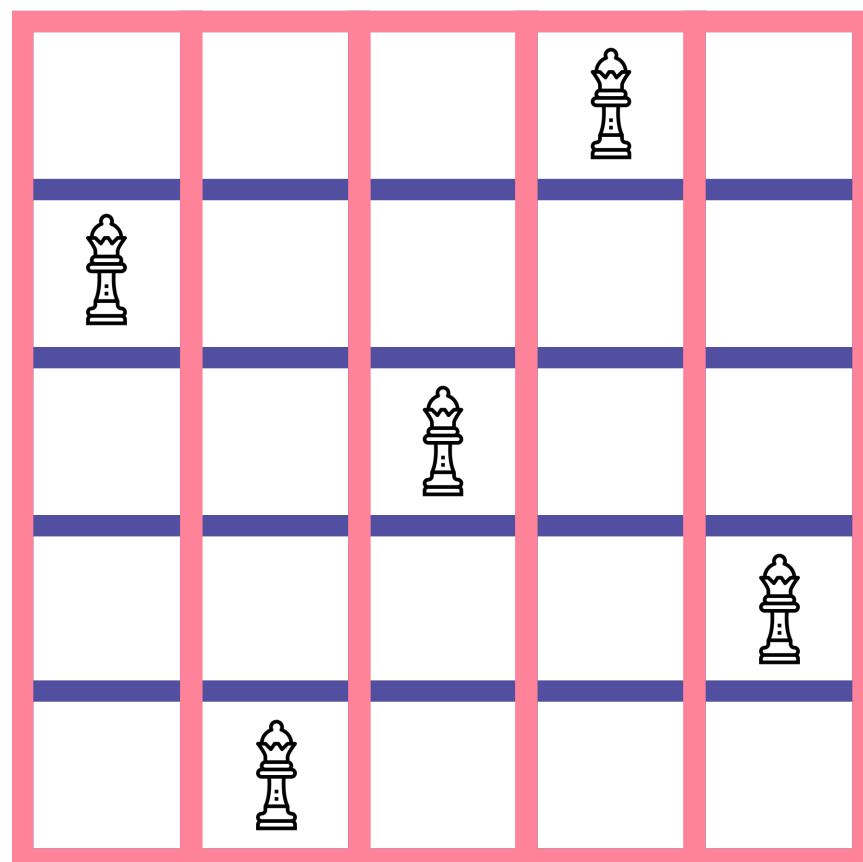


각 세로줄에 놓인 퀸이

위에서부터 몇 번째 칸에 있는지 한 번 세보면?



백트래킹 알고리즘 심화

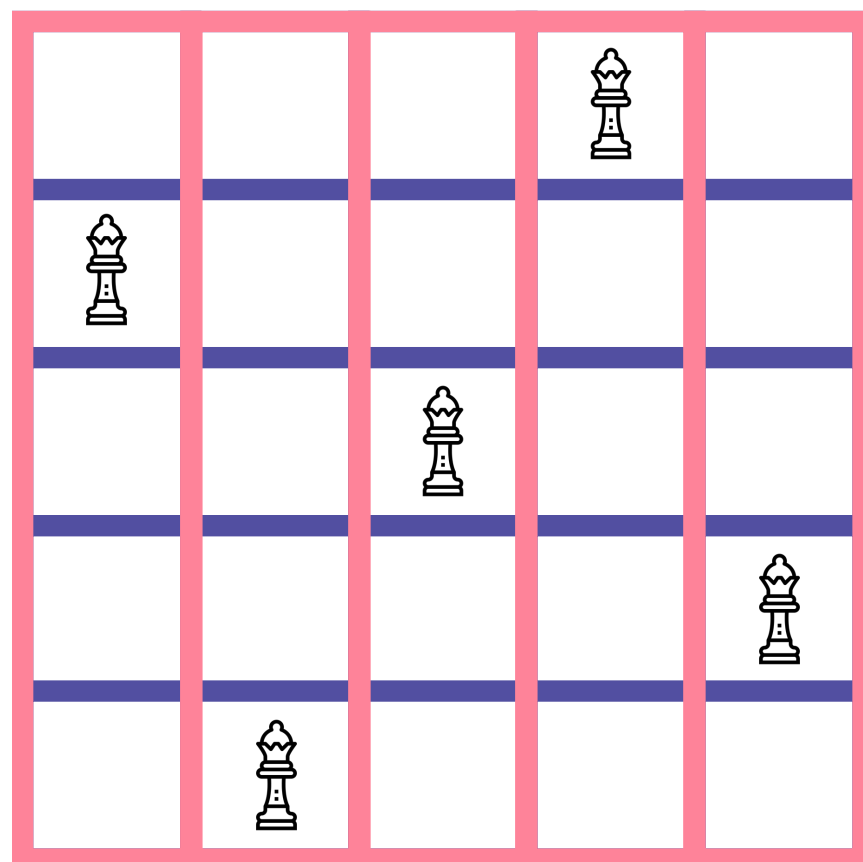


2 5 3 1 4

다음과 같이 적을 수 있음



백트래킹 알고리즘 심화

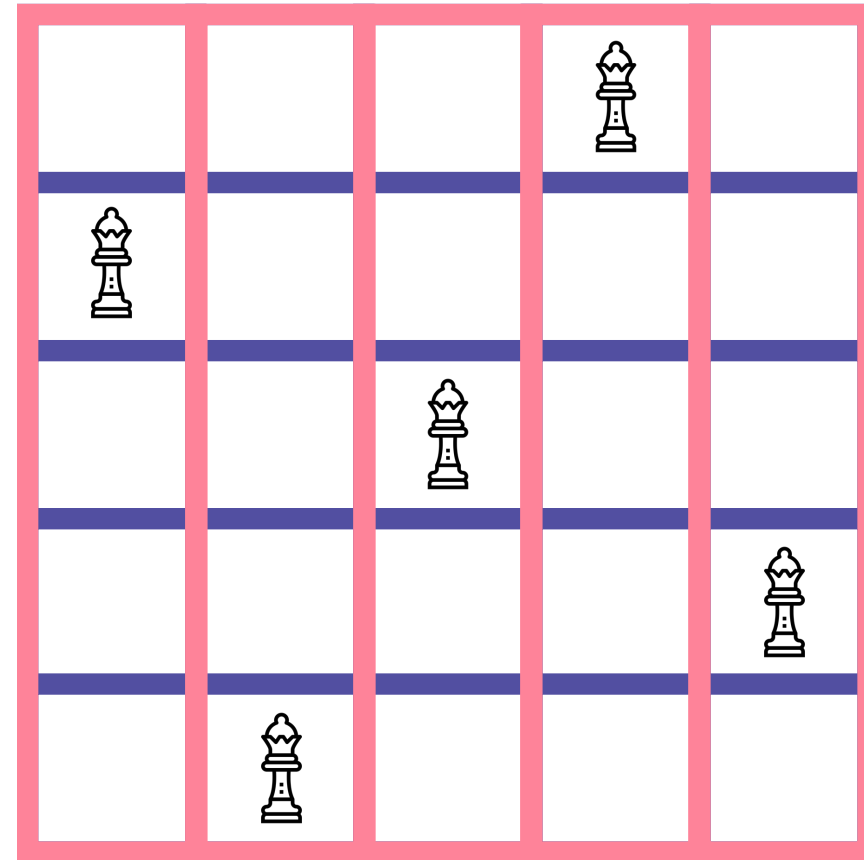


2 5 3 1 4

수를 잘 관찰해보면, 뭔가 수상한 것을 발견할 수 있음

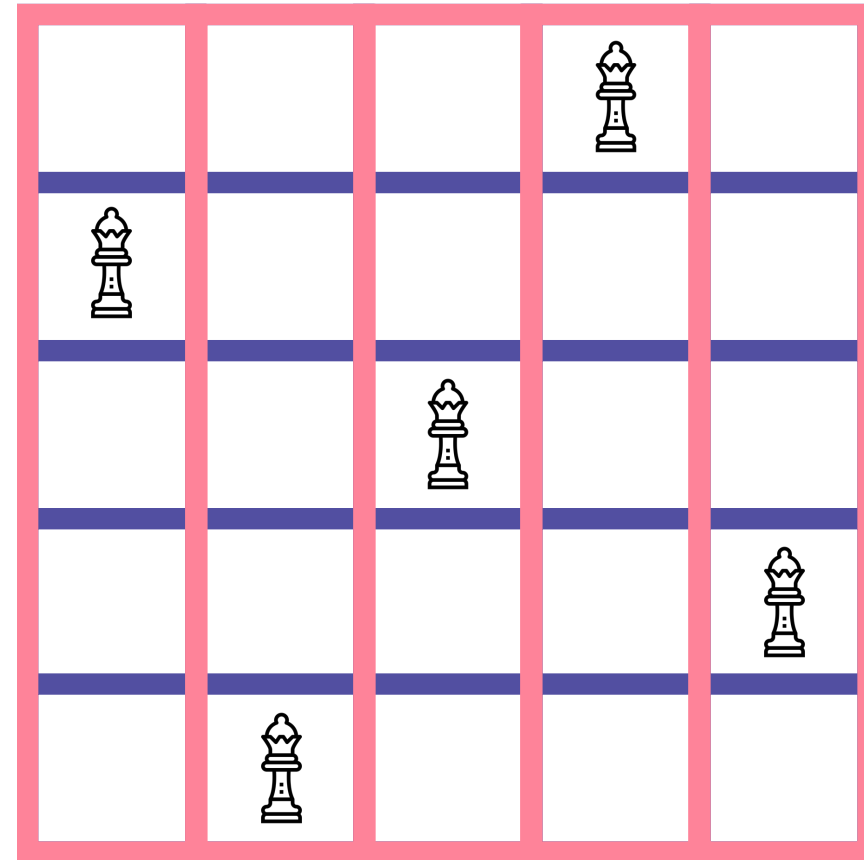


백트래킹 알고리즘 심화



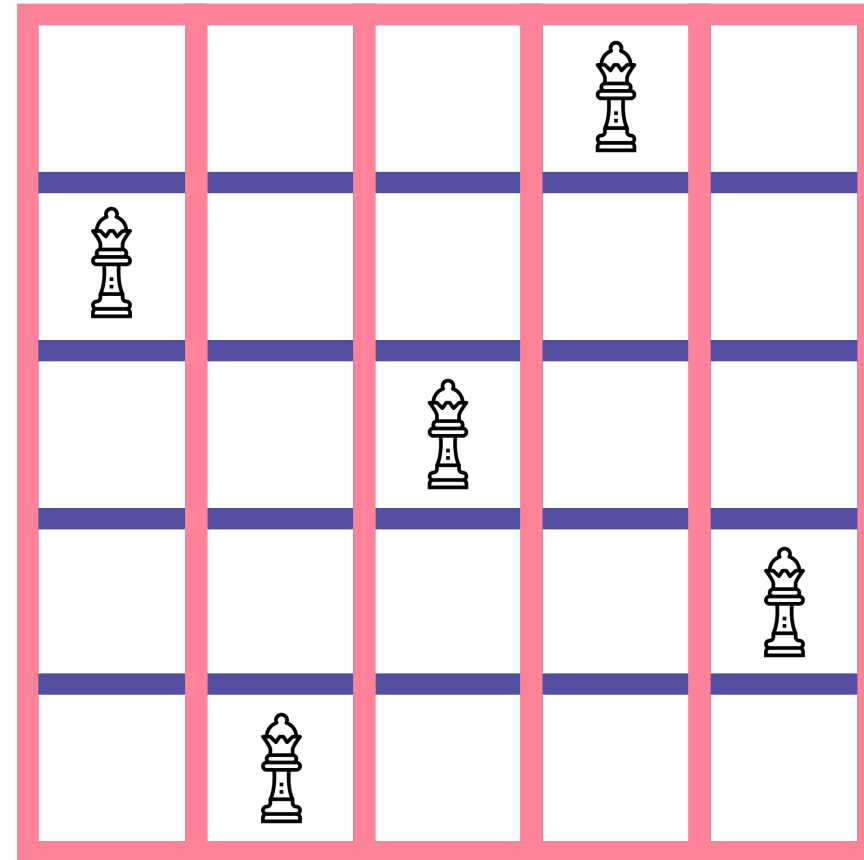
2 5 3 1 4

가로와 세로에서 만나는 두 퀸이 없어야 하기 때문에,
위와 같이 숫자를 적게 되면 같은 숫자가 없어야 함



2 5 3 1 4

즉, 앞에서 구한 순열과 동일한 것



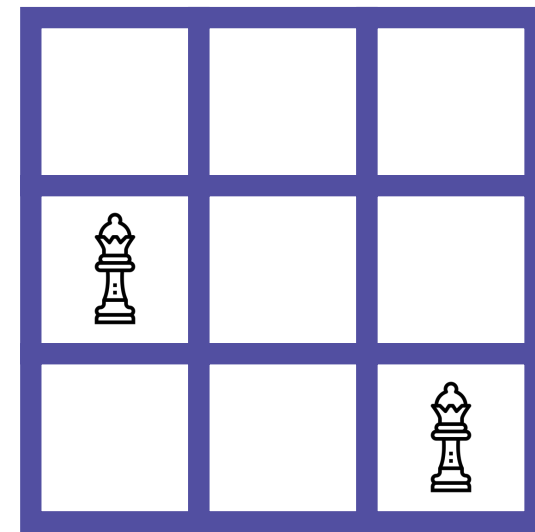
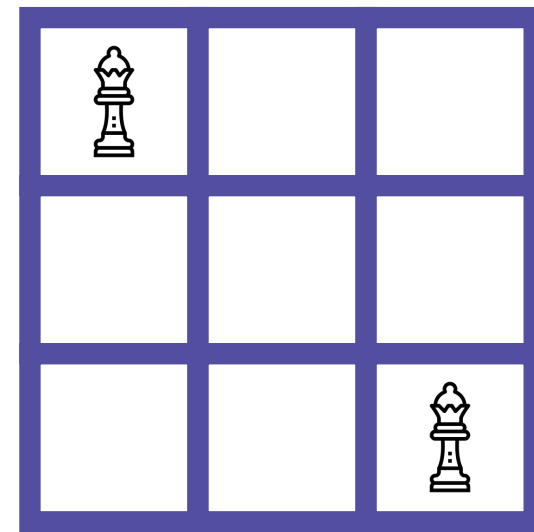
2 5 3 1 4

그렇지만, 이 문제는 여기서 끝이 아님

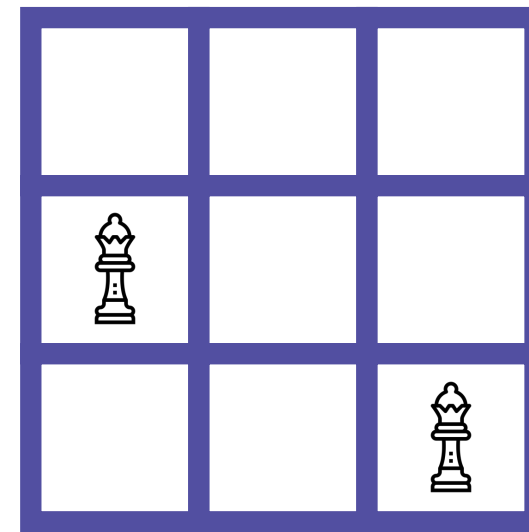
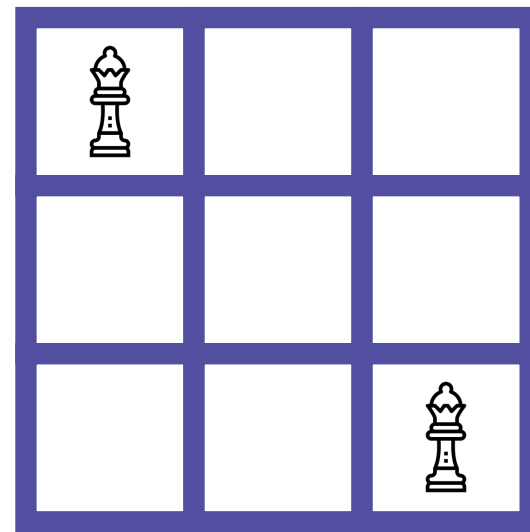
대각선을 고려해야 하기 때문



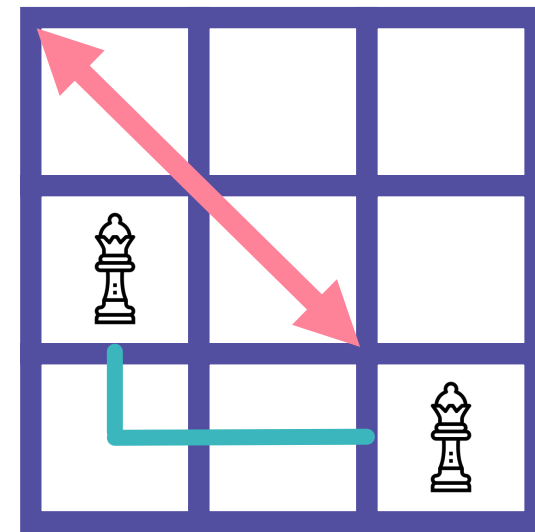
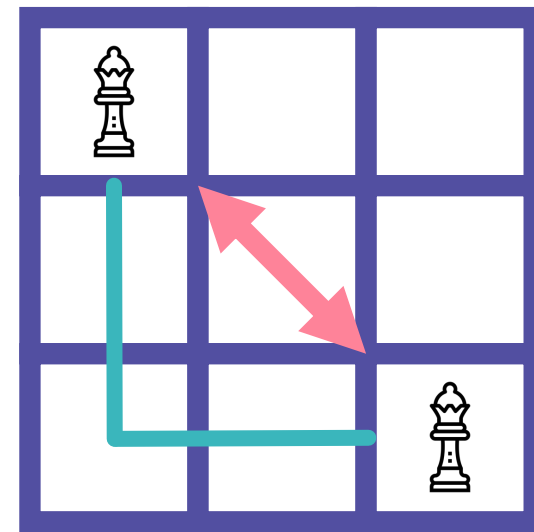
백트래킹 알고리즘 심화



대각선이 만나는 경우와, 만나지 않는 경우를 관찰해보자



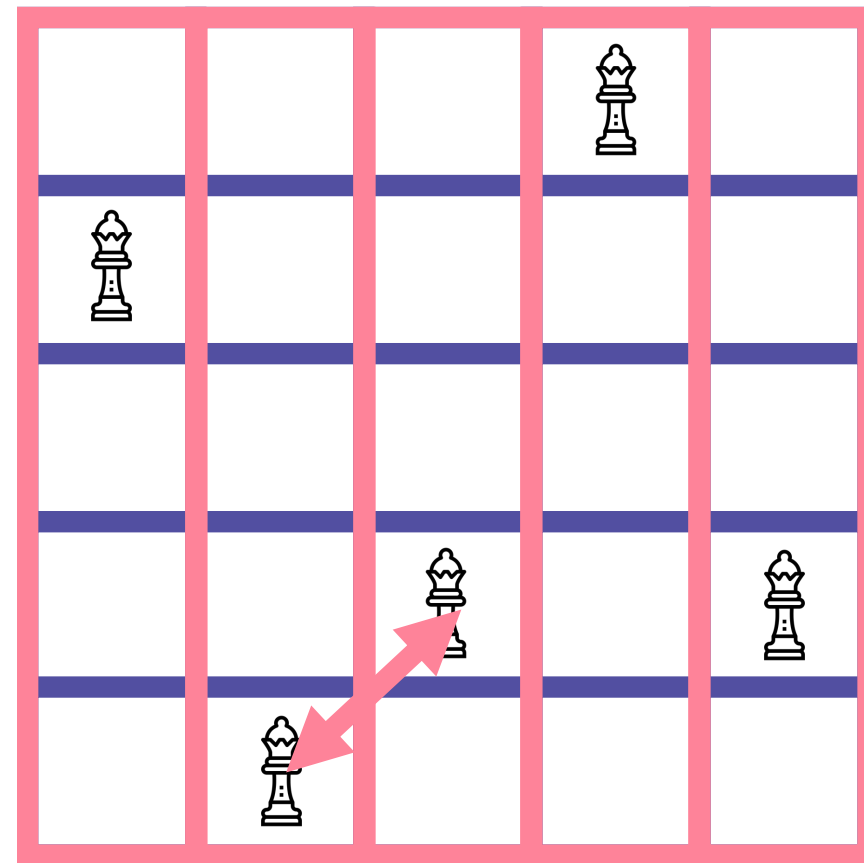
아직 감을 잘 못 잡겠다면, 가로와 세로의 차이를 중심으로
다시 한 번 살펴보자



대각선으로 만나는 경우는,
가로와 세로의 차이가 같음



백트래킹 알고리즘 심화



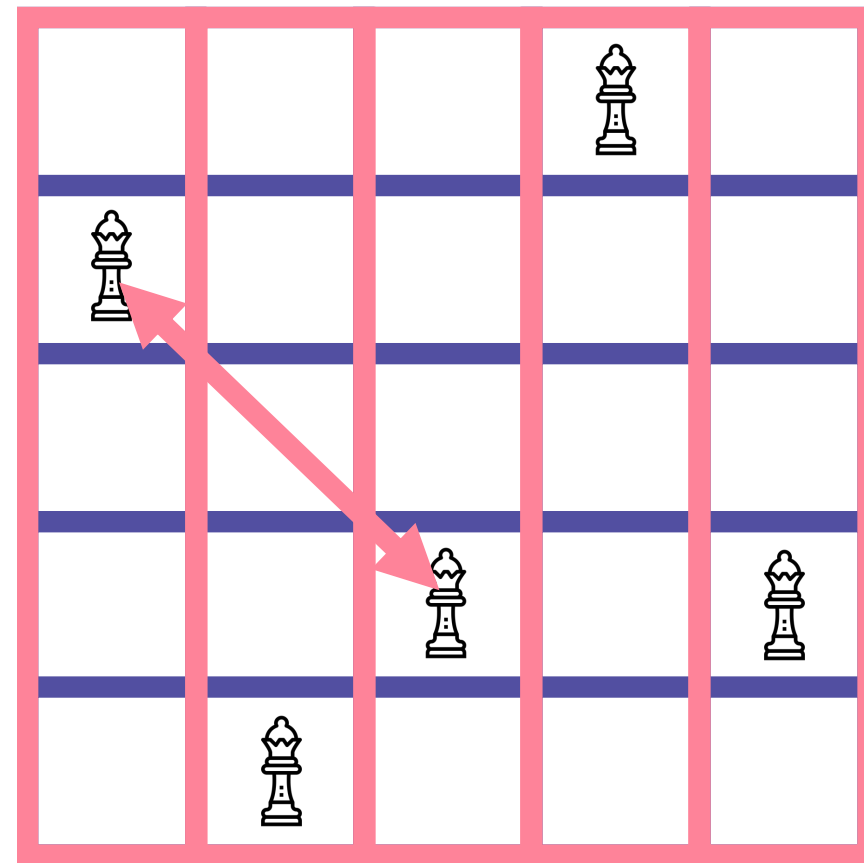
0	1	2	3	4
2	5	4	1	4

즉, $\text{인덱스(가로)의 차이} == \text{값(세로)의 차이}$ 라면,

대각선으로 만난다고 볼 수 있음



백트래킹 알고리즘 심화



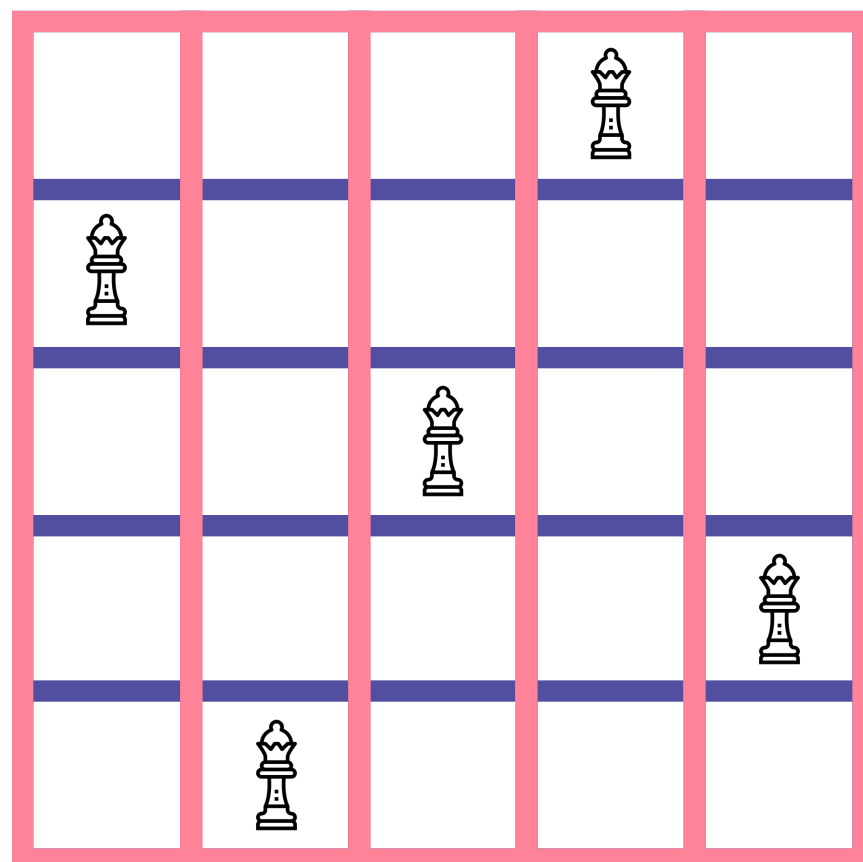
0	1	2	3	4
2	5	4	1	4

즉, $\text{인덱스(가로)의 차이} == \text{값(세로)의 차이}$ 라면,

대각선으로 만난다고 볼 수 있음



백트래킹 알고리즘 심화

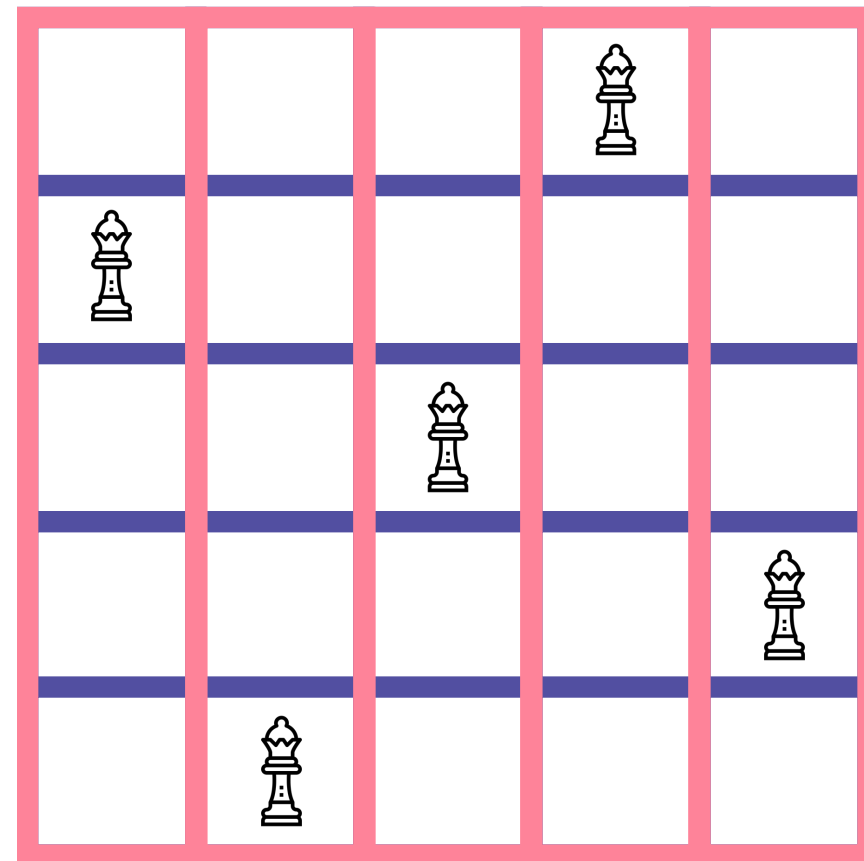


0	1	2	3	4
2	5	3	1	4

즉, **인덱스(가로)의 차이 == 값(세로)의 차이** 라면,
대각선으로 만난다고 볼 수 있음



백트래킹 알고리즘 심화



0	1	2	3	4
2	5	3	1	4

결국, 백트래킹을 통해 순열을 구하되,
대각선 조건($|i-j| = |a[i]-a[j]|$)을 추가해서
적절하게 가지치기를 해줘야 함



어려워보이는 문제여도, 결국은 기존에 사용했던 논리에서
크게 벗어나지 않거나, 약간 심화한 경우가 많음



그러므로 새로운 문제를 볼 때, 기존에 해결한 문제의 아이디어를
적용할 수 있는지 확인하면 도움이 되는 경우가 많음